UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Online FOOD Delivery System

Submitted by:

Name	SRN
Kanak Goyal	PES1UG23CS279
Kashish Mittal	PES1UG23CS290
K Riha Sagar	PES1UG23CS297
Khushee Kiran	PES1UG23CS303

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru - 560085, Karnataka, India

Problem Statement

Modern urban consumers demand rapid access to daily essentials — groceries, medicines, and household products — delivered within minutes. Traditional e-commerce platforms with multi-day delivery windows fail to meet this expectation. This project addresses the need for a Quick Commerce Delivery System (inspired by platforms like Zepto and Blinkit) that enables near-instant delivery of products from local stores to customers.

The system must handle end-to-end operations: product catalog management, customer cart and order processing, intelligent delivery partner dispatch, real-time order tracking, secure authentication, and wallet-based payments. The solution is implemented as a Java Spring Boot RESTful web application, designed using Object-Oriented Analysis and Design (OOAD) principles including SOLID principles, MVC architecture, and standard design patterns.

Key Features

Module 1 — Inventory & Catalog

- Real-time stock management with automatic availability toggling when stock reaches zero
- Low-stock alert system triggered when inventory falls below a configurable minimum threshold
- Product search by keyword with partial match support

- Category and subcategory-based product filtering

Module 2 — Order & Cart

- Cart management with live stock validation before adding items
- Price calculation engine: subtotal, 5% tax, configurable delivery charges, and discount application
- Support for multiple discount types: percentage-based, fixed amount, first-order, and seasonal offers
- Full checkout flow from cart to confirmed order with payment processing
- Complete order history per customer with one-click reorder functionality

Module 3 — Delivery & Tracking

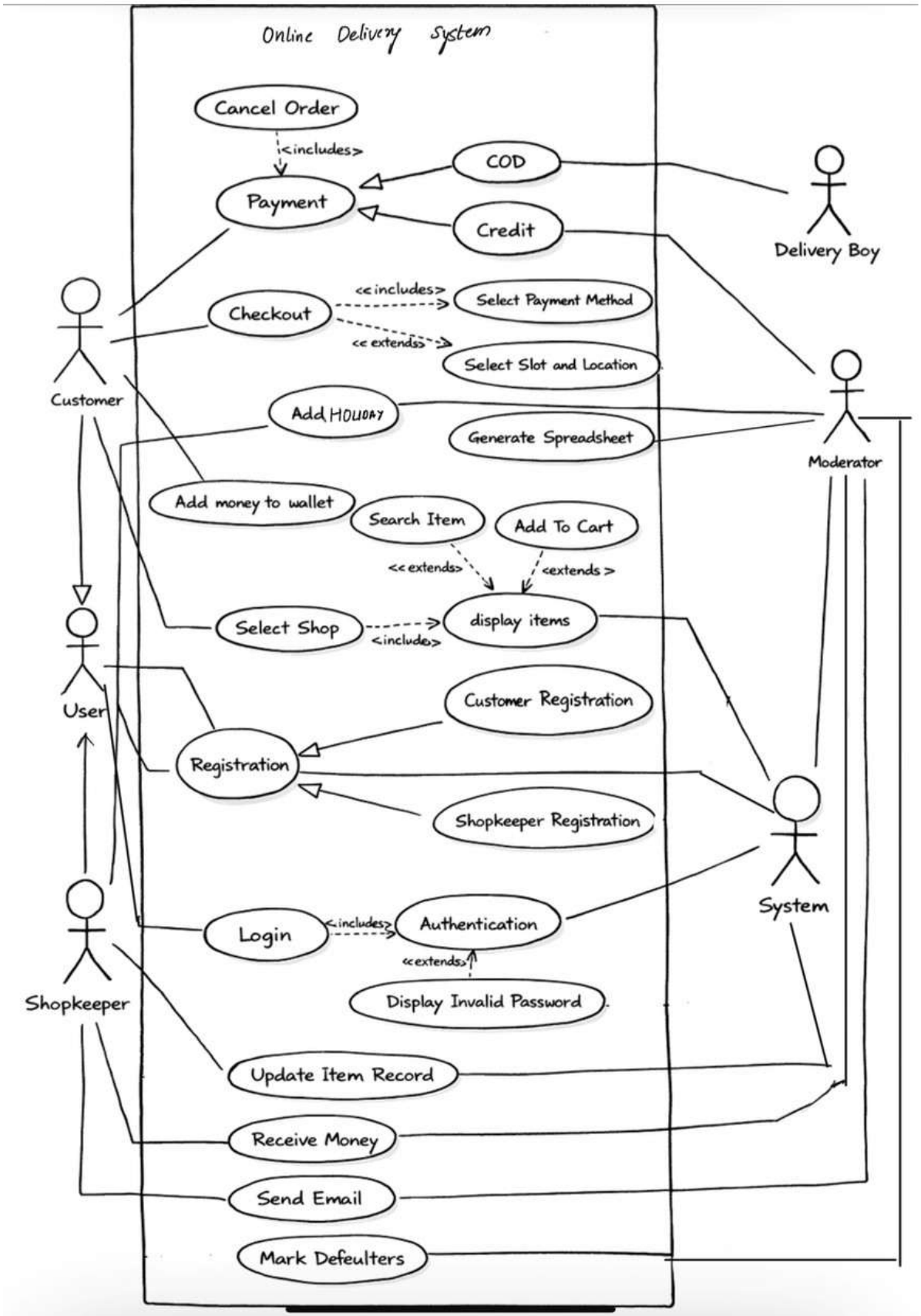
- Automatic delivery partner assignment based on availability
- ETA calculation using the Haversine formula to compute geodesic distance between store and customer
- Live order status progression: Packing → Out for Delivery → Delivered
- Real-time location broadcasting to the customer app via the Observer pattern • Delay detection with automatic customer notification

Module 4 — User & Profile

- Role-based access control (RBAC) supporting four roles: Customer, Delivery Partner, Store Manager, Admin
- JWT-based stateless authentication with Spring Security
- User Factory pattern for structured creation of different user types
- Wallet management: balance top-up, debit on purchase, and full transaction history
- Support for four payment methods: UPI, Card, Cash on Delivery (COD), and Wallet

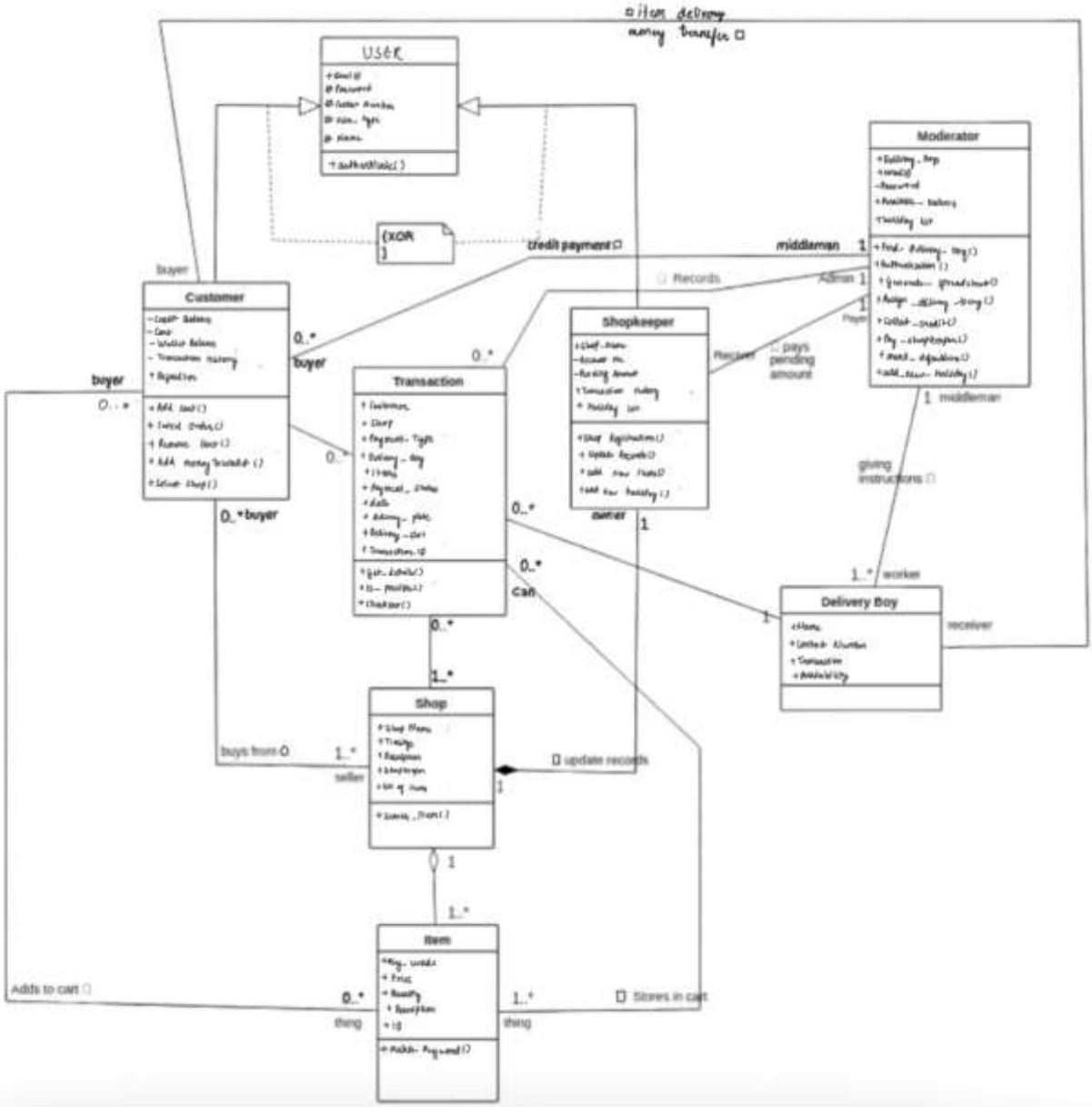
Models

Use Case Diagram


Online Delivery system
Cancel Order
<includes>
COD
Payment
Credit
Delivery Boy
Checkout
<<includes>
Select Payment Method
<<extends>
Select Slot and Location
Customer
Add HOLIDAY
Generate Spreadsheet
Moderator
Add money to wallet
Search Item
Add To Cart
<<extends>
<extends>
Select Shop
display items
<include>
User
Customer Registration
Registration
Shopkeeper Registration
System
Login
<includes>
Authentication
<<extends>
Shopkeeper
Display Invalid Password
Update Item Record
Receive Money
Send Email
Mark Defeulters

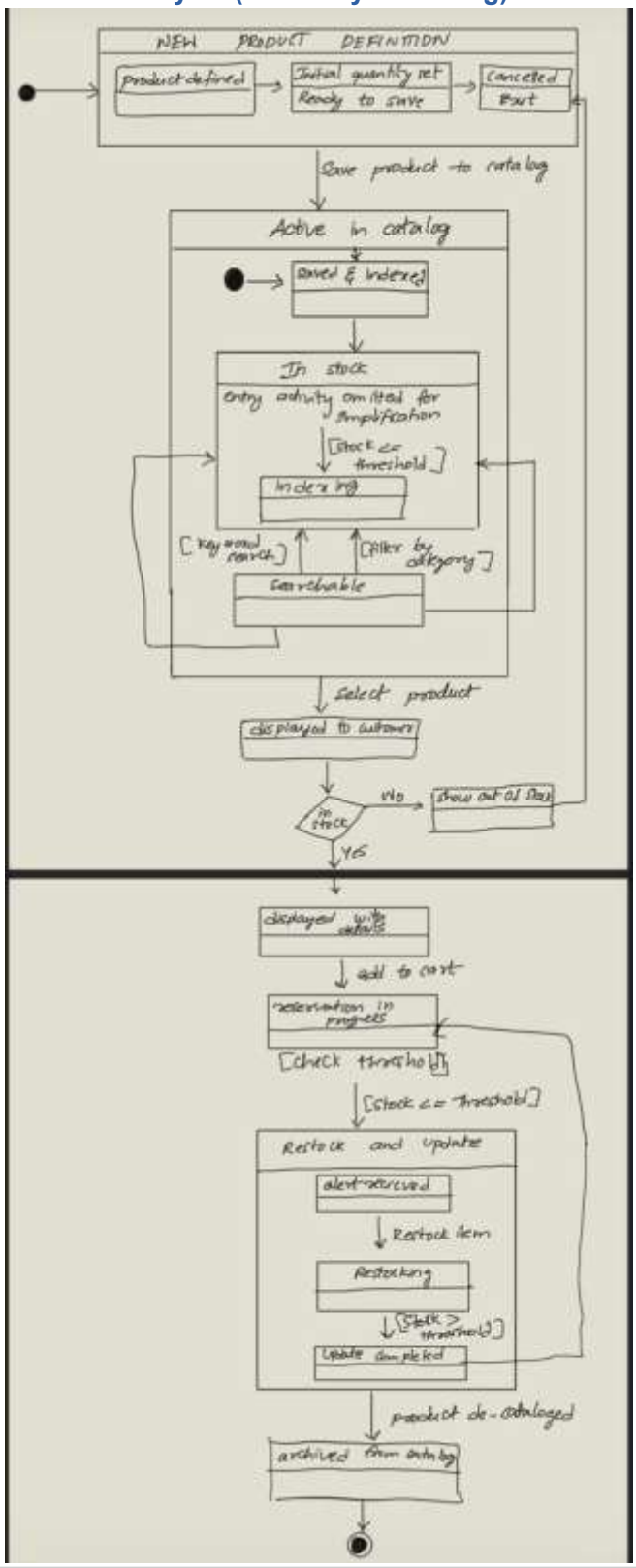
Class Diagram

2. Class Diagram

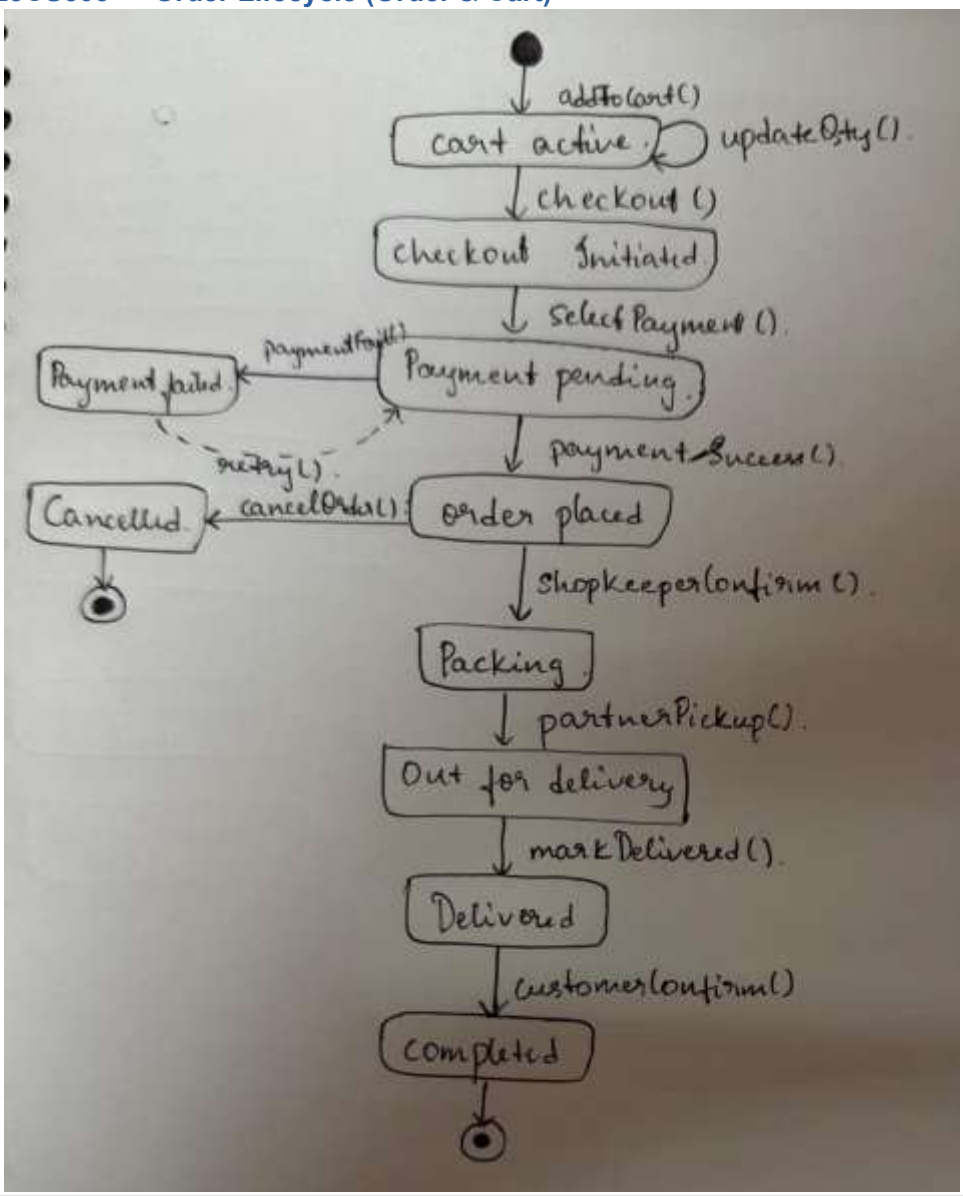


State Diagrams

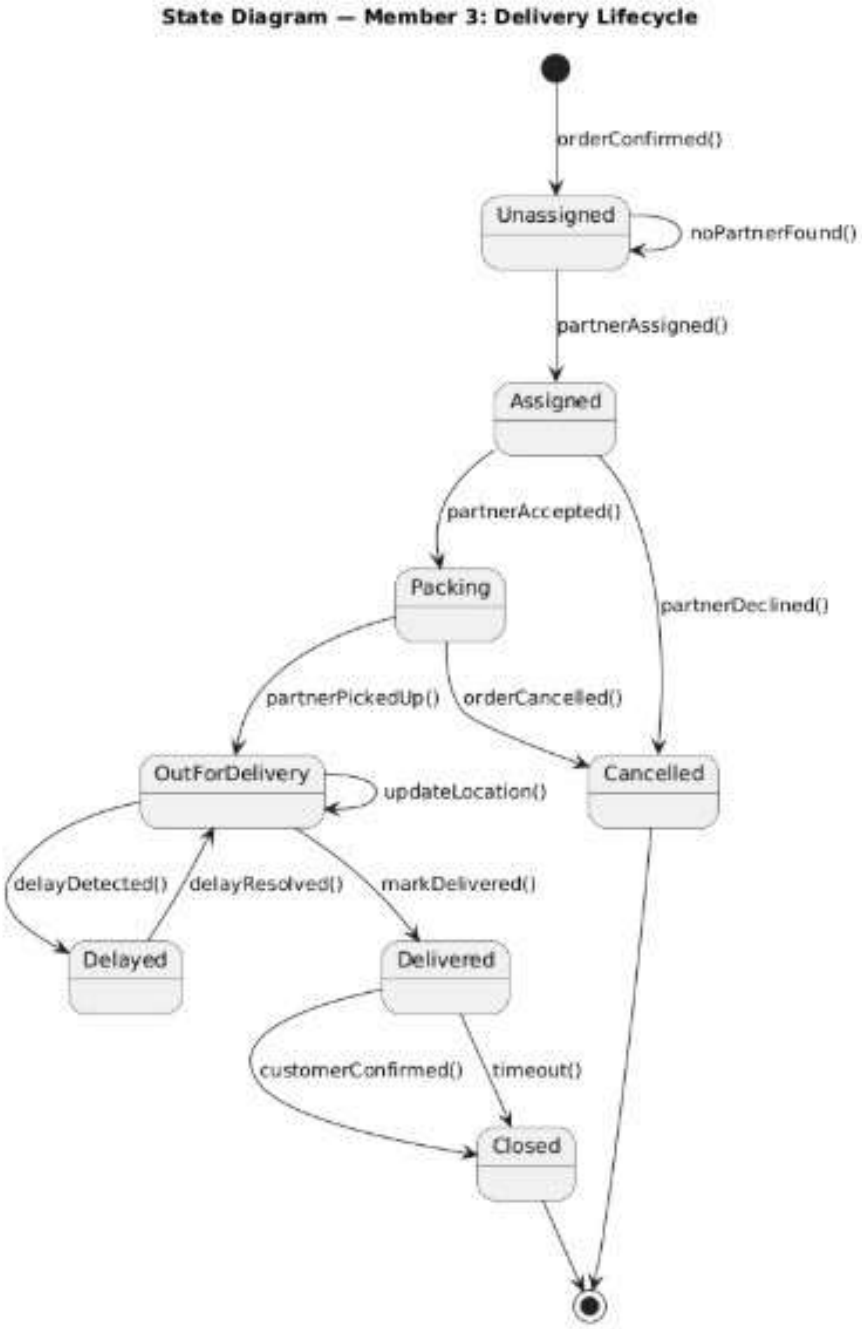
1 PES1UG23CS297 — Product Lifecycle (Inventory & Catalog)



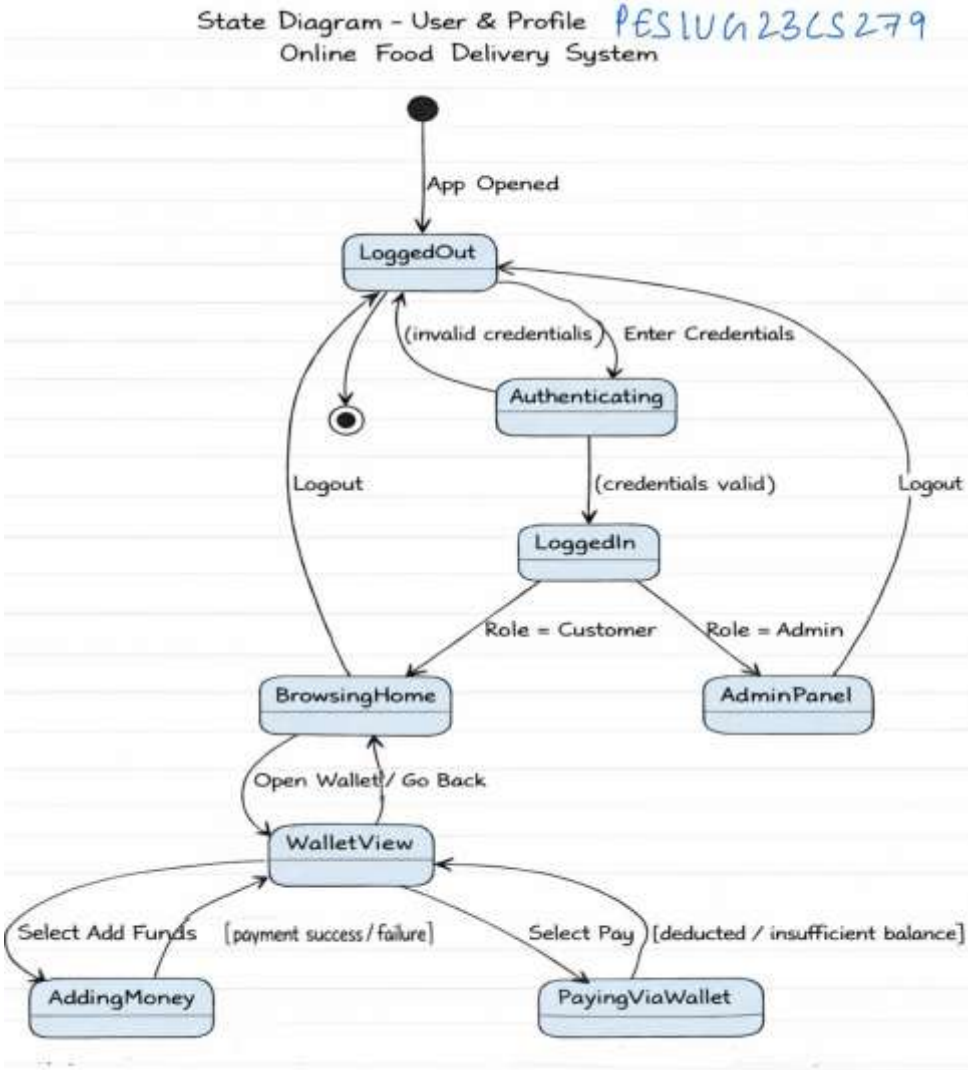
2 PES1UG23CS303 — Order Lifecycle (Order & Cart)



3 PES1UG23CS290 — Delivery Lifecycle (Delivery & Tracking)

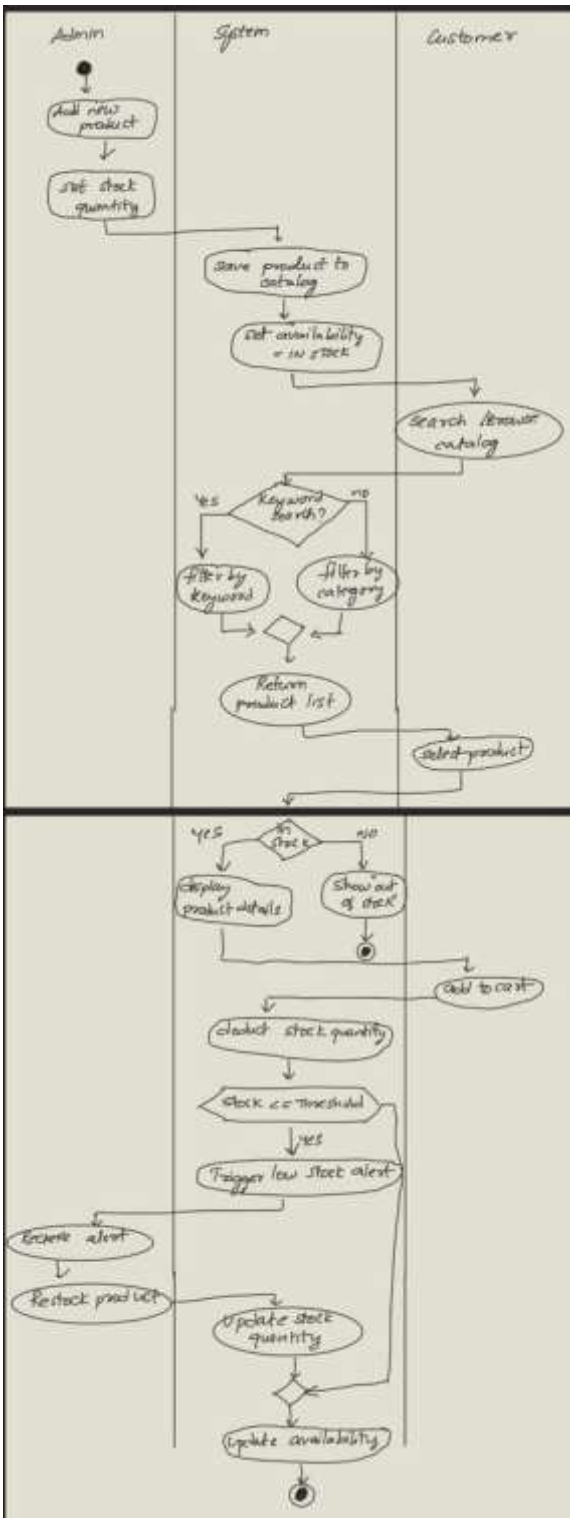


4 PES1UG23CS279— User Lifecycle (User & Profile)

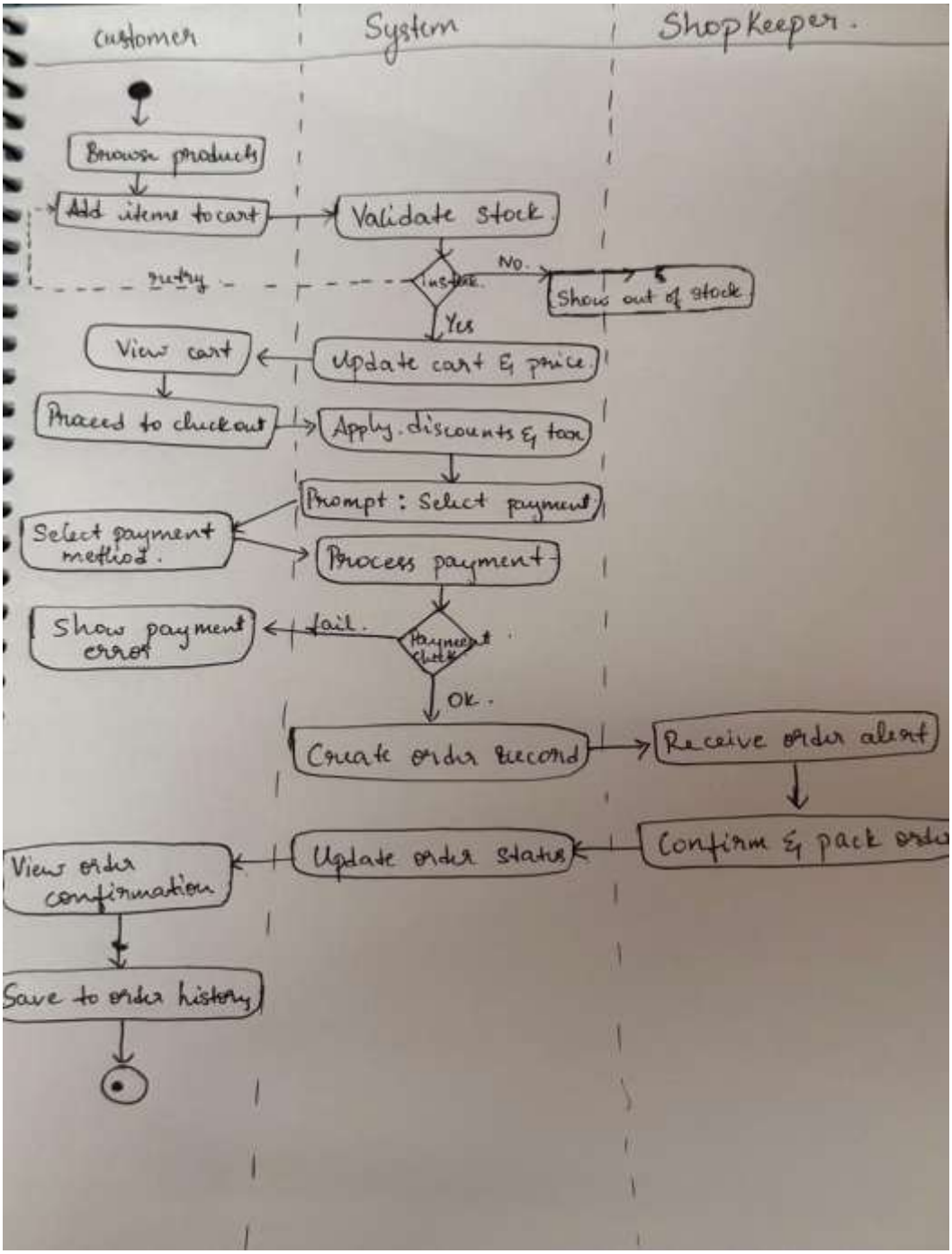


Activity Diagrams

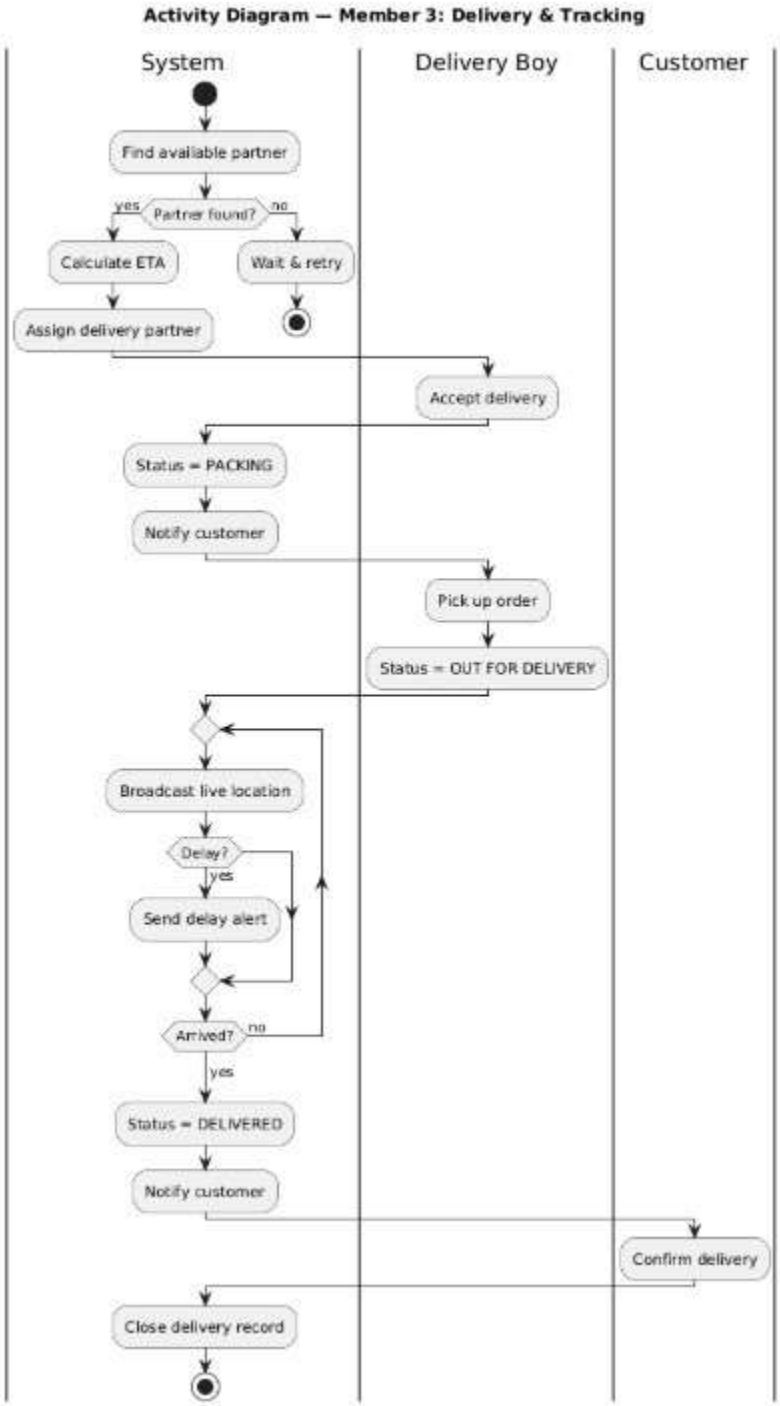
1 PES1UG23CS297— Inventory & Catalog



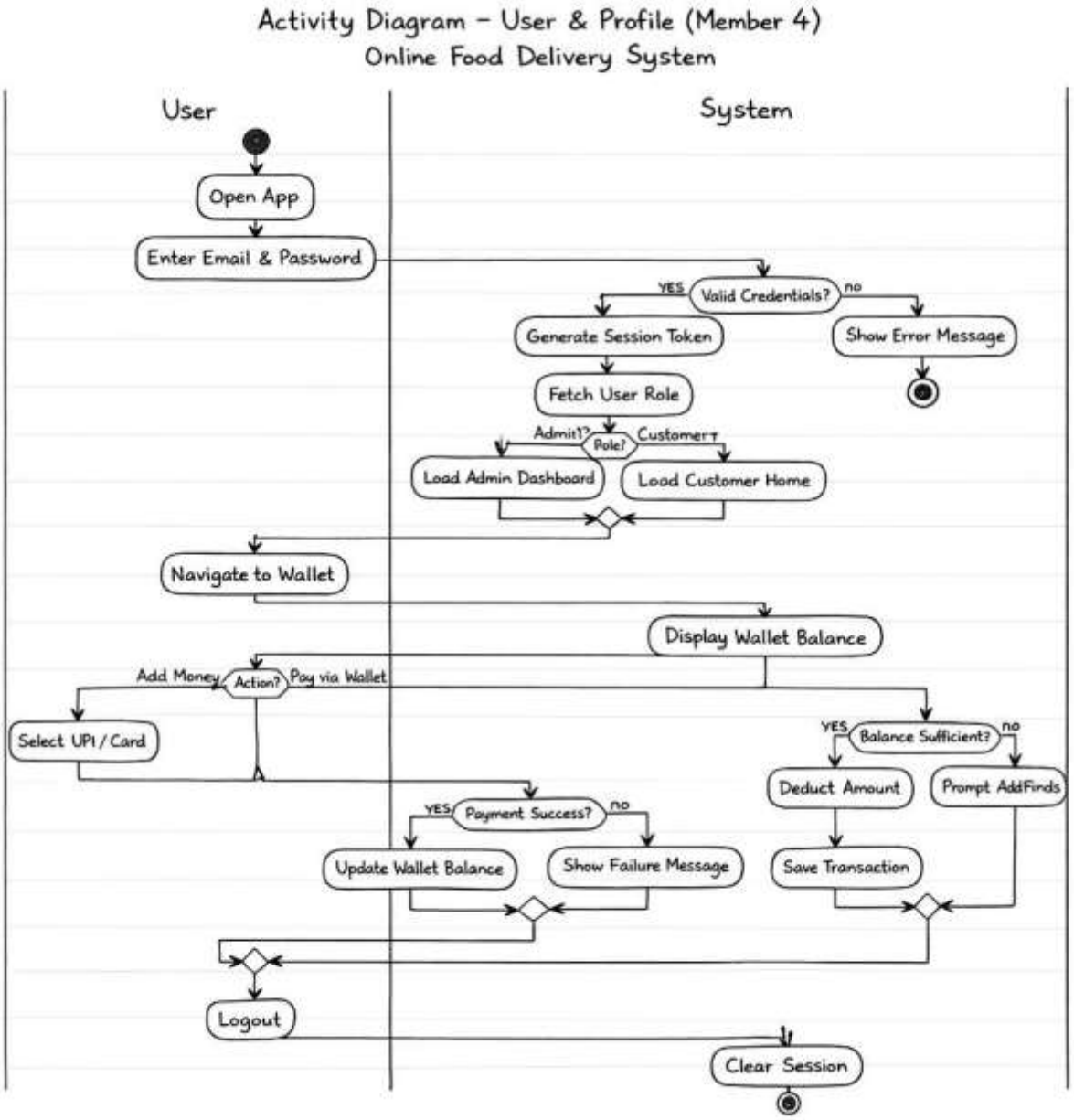
2 PES1UG23CS303— Order & Cart



3 PES1UG23CS290— Delivery & Tracking



4 PES1UG23CS279 — User & Profile



Design Principles and Design Patterns

MVC Architecture

Yes — the application strictly follows the Model-View-Controller (MVC) pattern as implemented by Spring Boot.

- Model: JPA entity classes (Order, Product, User, Delivery, Cart, WalletTransaction, etc.) represent the data layer and map directly to MySQL database tables.
- View: Thymeleaf HTML templates (home.html, products.html, cart.html, orders.html, deliveries.html, login.html, register.html) render the user interface.

- Controller: REST controllers (OrderController, ProductController, CartController, DeliveryController, UserController, WalletController) handle HTTP requests and delegate business logic to the service layer.

Design Principles

Single Responsibility Principle (SRP)

Each service class is responsible for exactly one business domain. OrderService handles only order operations; PaymentService handles only payment processing; WalletService handles only wallet transactions; DeliveryService handles only delivery assignment and tracking. No service class crosses into another domain's concerns.

Open/Closed Principle (OCP)

The payment system is open for extension but closed for modification. New payment methods can be added by implementing the PaymentStrategy interface without touching existing strategy classes. Similarly, new discount types can be added to the DiscountType enum and handled in the discount logic without modifying existing discount code. New user roles can be added to UserFactory by adding a new case to the switch statement.

Liskov Substitution Principle (LSP)

All implementations of PaymentStrategy (UPIPaymentStrategy, CardPaymentStrategy, CODPaymentStrategy, WalletPaymentStrategy) are interchangeable at runtime. The PaymentService operates only on the PaymentStrategy interface, so any concrete strategy can substitute another without breaking the calling code. Similarly, all OrderObserver implementations can be attached to the OrderSubject and will be called uniformly.

Interface Segregation Principle (ISP)

The PaymentStrategy interface exposes only payment-relevant methods (processPayment, validatePaymentDetails, getPaymentMethodName, supportsRefund). The OrderObserver interface exposes only observer-relevant methods (update, getObserverName). No client is forced to depend on methods it does not use.

Dependency Inversion Principle (DIP)

Controllers depend on service abstractions, not concrete implementations. Services depend on Spring Data JPA repository interfaces, not database-specific implementations. The PaymentService depends on the PaymentStrategy interface, not on any specific payment class. The Observer pattern in OrderService depends on the OrderObserver interface, decoupling the notification mechanism from concrete observers.

Design Patterns

1. Factory Method Pattern (Creational)

File: com.quickcommerce.patterns.factory.UserFactory

The UserFactory class provides a static factory method createUser(UserRole, ...) that instantiates the correct type of User object based on the role parameter. This removes the responsibility of user construction from the calling code and ensures that each role is initialised with the correct default values (e.g., delivery partners start with isActive=false pending verification; admins start with isVerified=true). Adding a new user role requires only a new case in the factory switch, not changes to any caller.

2. Proxy Pattern (Structural)

Files: com.quickcommerce.patterns.proxy.Image, RealImage, ProxyImage

The ProxyImage class implements the Image interface and wraps a RealImage. The actual image file is loaded from disk only when display() is called for the first time (lazy loading). On subsequent calls the cached image is returned. This reduces startup time and memory usage when the product catalog contains many high-resolution images, as images are only loaded when a user actually views the product.

3. Strategy Pattern (Behavioral)

Files: com.quickcommerce.patterns.strategy.PaymentStrategy and its implementations

The PaymentStrategy interface defines a contract for processing payments. Four concrete strategies are implemented: UPIPaymentStrategy (validates UPI ID and PIN), CardPaymentStrategy (validates card number, CVV, and expiry), CODPaymentStrategy (always succeeds at order placement, no pre-validation), and WalletPaymentStrategy (checks wallet balance before deducting). The PaymentService selects the appropriate strategy at runtime based on the payment method chosen during checkout, allowing the payment mechanism to be swapped without changing any order logic.

4. Observer Pattern (Behavioral)

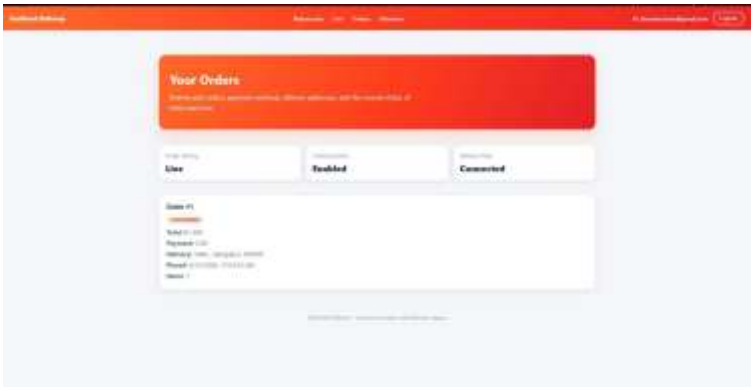
Files: com.quickcommerce.patterns.observer.OrderObserver, OrderSubject, CustomerAppObserver, DeliveryPartnerAppObserver

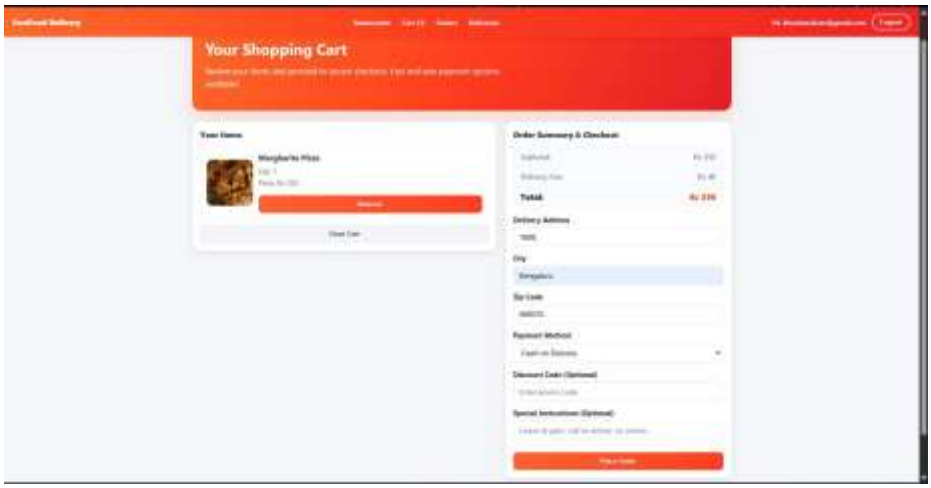
The OrderSubject maintains a list of OrderObserver instances. When an order status changes (e.g., from PENDING to PACKING, or from PACKING to OUT_FOR_DELIVERY), the OrderService calls notifyObservers() on the subject. Each attached observer then fires its own update() logic: CustomerAppObserver sends a push notification to the customer, and DeliveryPartnerAppObserver notifies the assigned delivery partner. New observers (e.g., an SMS notification service) can be attached without modifying OrderService.

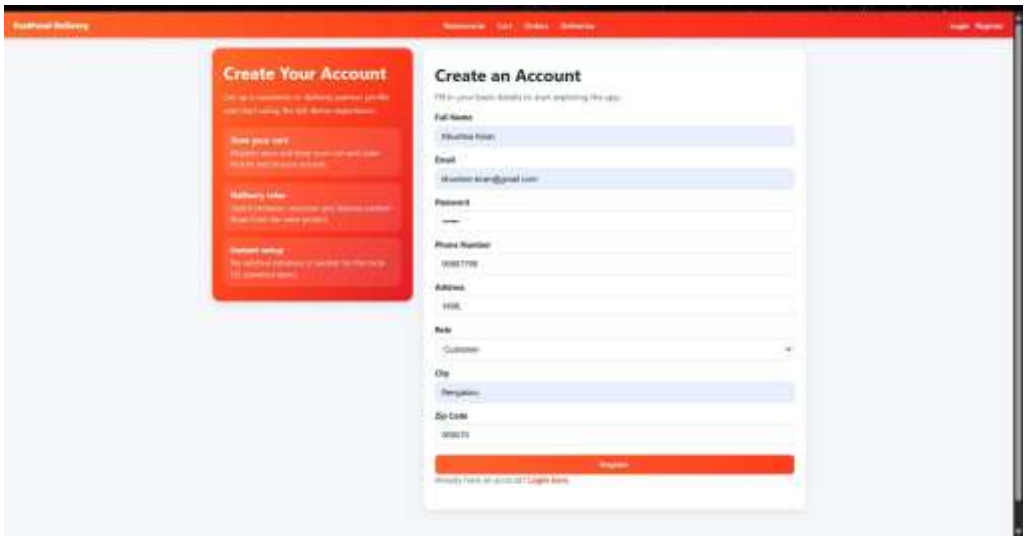
GitHub link to the codebase: https://github.com/kookiepachi/ooad_miniproject

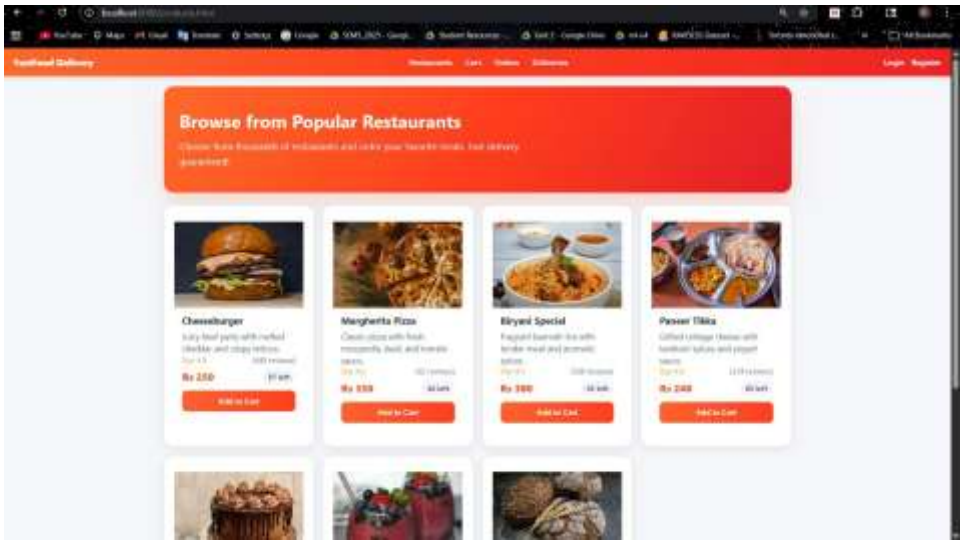
Screenshots

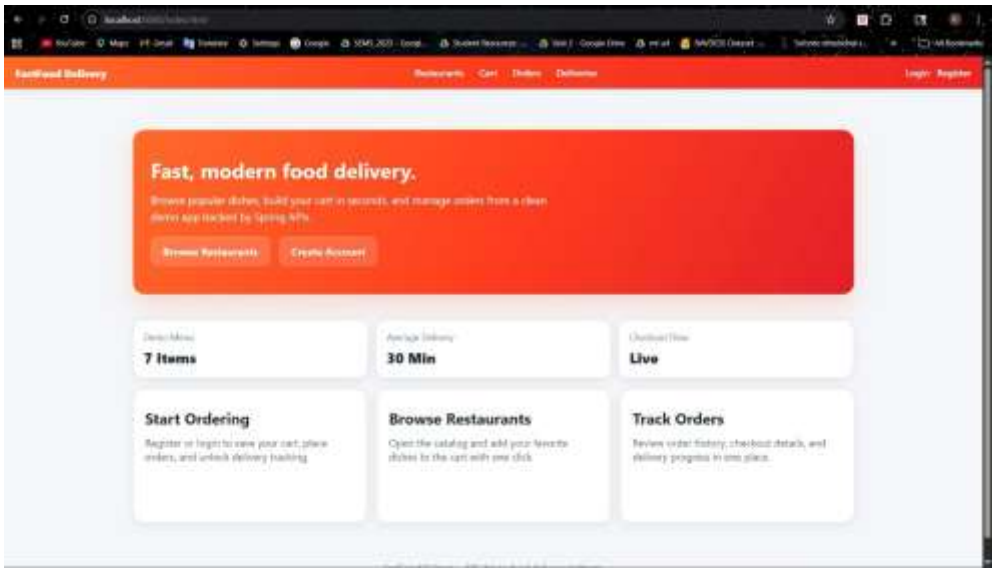
UI




Your Shopping Cart


Create Your Account
Create an Account


Browse from Popular Restaurants
Cheeseburger
Margherita Pizza
Biryani Special
Paneer Tikka



Individual Contributions of Team Members

Name	Module	Features Implemented
Riha Sagar	Module 1 — Inventory & Catalog	Stock management, low-stock alerts, product search & category filtering
Khushee Kiran	Module 2 — Order & Cart	Cart logic, price calculation, checkout flow, order history, reorder
Kashish Mittal	Module 3 — Delivery & Tracking	Partner assignment, ETA calculation, live status updates, delay detection
Kanak Goyal	Module 4 — User & Profile	JWT authentication, RBAC, UserFactory, wallet & payment management